# Task 07 - Daily Workflow in a Real Repository

Handbook reference: Page 7 of 10 | Report date: 2026-10-13 | Folder: `task-07-repo-daily-workflow/`

## 1. Concept from the handbook

Before using AI in a repository, understand its state: existing uncommitted changes may belong to someone else and must not be overwritten or silently included. Flow: instructions + status -> code + criteria -> branch/worktree -> small change -> inspect diff -> checks -> PR review -> exact revision. Local pass, CI pass, merged and deployed are different states.

## 2. What was built

Real **git** automation: **preflight** (finds AGENTS.md, branch, foreign uncommitted files), **branch creation that refuses a dirty tree**, **diff inspection** with out-of-scope file detection, check runner, and a **handoff** that records the exact commit and an enum of delivery states - listing which ones are evidenced and which are not.

## 3. Behaviours demonstrated

- Dirty tree -> refuse to branch, report who-knows-whose changes
- Diff outside allowed scope is flagged
- Handoff refuses to run on a dirty tree (evidence must match one exact revision)
- Only LOCAL_TESTS_PASS can be claimed; CI / merged / deployed are shown as NOT evidenced

## 4. Files in this task

`demo.py`, `repo_workflow.py`, `test_repo_workflow.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

## 5. Code screenshots

repo_workflow.py

```python
"""Task 07 - Daily workflow in a REAL git repository.

1 instructions + status -> 2 code + criteria -> 3 branch -> 4 small change
5 inspect diff -> 6 checks -> 7 PR review -> 8 exact revision

Everything here shells out to real `git`. It never overwrites uncommitted work
that isn't yours; it reports it instead.
"""
from __future__ import annotations

import subprocess
import sys
from dataclasses import dataclass, field
from enum import Enum
from pathlib import Path
from typing import List, Optional

INSTRUCTION_FILES = ["AGENTS.md", "CLAUDE.md", "CONTRIBUTING.md"]


def git(repo: Path, *args: str) -> str:
    cp = subprocess.run(["git", *args], cwd=repo, capture_output=True, text=True)
    if cp.returncode != 0:
        raise RuntimeError(f"git {' '.join(args)} failed: {cp.stderr.strip()}")
    return cp.stdout.strip()


@dataclass
class Preflight:
    instructions: List[str]
    branch: str
    uncommitted: List[str]          # someone else's work - do NOT overwrite / silently include
    clean: bool


def preflight(repo: Path) -> Preflight:
    instr = [f for f in INSTRUCTION_FILES if (repo / f).exists()]
    status = git(repo, "status", "--porcelain")
    changed = [l[3:] for l in status.splitlines()]
    return Preflight(instr, git(repo, "rev-parse", "--abbrev-ref", "HEAD"), changed, not changed)


def create_branch(repo: Path, name: str) -> None:
    pf = preflight(repo)
    if not pf.clean:
        raise RuntimeError(f"refusing to branch with uncommitted changes that are not ours: {pf.uncommitted}")
    git(repo, "checkout", "-b", name)


@dataclass
class DiffReport:
    files: List[str]
    insertions: int
    deletions: int
    out_of_scope: List[str]

    @property
    def bounded(self) -> bool:
        return not self.out_of_scope


def inspect_diff(repo: Path, base: str, allowed: List[str]) -> DiffReport:
    names = git(repo, "diff", "--name-only", base).splitlines()
    ins = dele = 0
    for line in git(repo, "diff", "--numstat", base).splitlines():
        a, d, _ = line.split("\t", 2)
        ins += int(a) if a.isdigit() else 0
        dele += int(d) if d.isdigit() else 0
    return DiffReport(names, ins, dele, [n for n in names if n not in allowed])


def run_checks(repo: Path, commands: List[List[str]]) -> List[dict]:
    out = []
    for cmd in commands:
        cp = subprocess.run(cmd, cwd=repo, capture_output=True, text=True)
        out.append({"cmd": " ".join(["python" if cmd[0] == sys.executable else cmd[0], *cmd[1:]]),
                    "rc": cp.returncode,
                    "tail": (cp.stdout + cp.stderr).strip().splitlines()[-1:] or [""]})
    return out

```

Main implementation: repo_workflow.py (part 1/2)

```python

class State(Enum):
    LOCAL_TESTS_PASS = "Local tests pass: selected checks pass on the tested revision"
    HOSTED_CI_PASS = "Hosted CI pass: CI checked this specific revision/config"
    MERGED = "Merged: integrated into base branch"
    DEPLOYED = "Deployed: released to a runtime; behaviour still needs observing"


@dataclass
class Handoff:
    revision: str
    checks: List[dict]
    states: List[State]
    not_run: List[str]
    limitations: List[str] = field(default_factory=list)

    def render(self) -> str:
        lines = [f"Exact revision tested: {self.revision}", "Checks run:"]
        lines += [f"  - `{c['cmd']}` -> rc={c['rc']} ({c['tail'][0]})" for c in self.checks]
        lines.append("Delivery states EVIDENCED: " + (", ".join(s.name for s in self.states) or "none"))
        lines.append("Delivery states NOT evidenced: " +
                     ", ".join(s.name for s in State if s not in self.states))
        lines.append("Checks not run: " + ("; ".join(self.not_run) or "none"))
        lines.append("Limitations: " + ("; ".join(self.limitations) or "none stated"))
        return "\n".join(lines)


def make_handoff(repo: Path, checks: List[dict], not_run: List[str], limitations: Optional[List[str]] = None) -> Handc
    if git(repo, "status", "--porcelain"):
        raise RuntimeError("working tree is dirty: evidence would not belong to a single exact revision")
    rev = git(repo, "rev-parse", "--short", "HEAD")
    states = [State.LOCAL_TESTS_PASS] if checks and all(c["rc"] == 0 for c in checks) else []
    return Handoff(rev, checks, states, not_run, limitations or [])
```

Main implementation: repo_workflow.py (part 2/2)

demo.py

```python
import sys, tempfile
from pathlib import Path
from repo_workflow import create_branch, git, inspect_diff, make_handoff, preflight, run_checks

repo = Path(tempfile.mkdtemp())
git(repo, "init", "-q", "-b", "main"); git(repo, "config", "user.email", "intern@example.com"); git(repo, "config", "us
(repo / ".gitignore").write_text("__pycache__/\n")
(repo / "AGENTS.md").write_text("Run: python -m unittest\nNo unrelated refactors.\n")
(repo / "tracker.py").write_text('def filter_tasks(tasks, status="all"):\n    return list(tasks)\n')
(repo / "test_tracker.py").write_text("import unittest\nfrom tracker import filter_tasks\n\nclass T(unittest.TestCase):
git(repo, "add", "."); git(repo, "commit", "-qm", "baseline")

print("1) Instructions + status  (someone else left uncommitted work)")
(repo / "colleague_notes.txt").write_text("wip")
pf = preflight(repo)
print(f"   instructions={pf.instructions} branch={pf.branch} clean={pf.clean} uncommitted={pf.uncommitted}")
try:
    create_branch(repo, "feature/status-filter")
except RuntimeError as e:
    print("   REFUSED:", e)
(repo / "colleague_notes.txt").unlink()           # (colleague finished; tree is clean again)

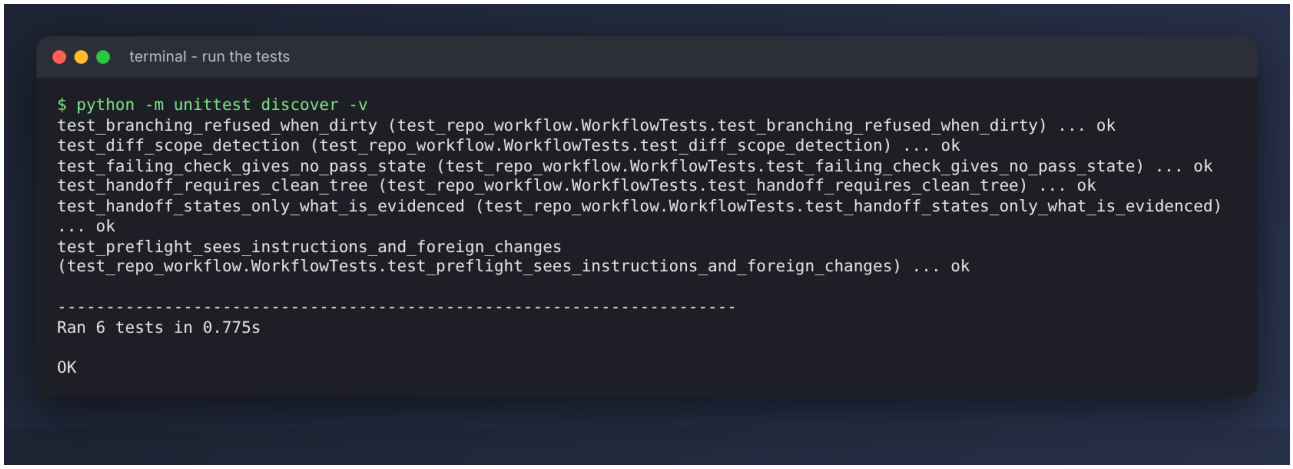
print("\n3) Branch")
create_branch(repo, "feature/status-filter"); print("   on", preflight(repo).branch)

print("\n4) Small change")
(repo / "tracker.py").write_text('def filter_tasks(tasks, status="all"):\n    if status == "all":\n        return list(
print("5) Inspect diff")
d = inspect_diff(repo, "HEAD", allowed=["tracker.py", "test_tracker.py"])
print(f"   files={d.files} +{d.insertions}/-{d.deletions} bounded={d.bounded}")
git(repo, "commit", "-qam", "Add status filter")

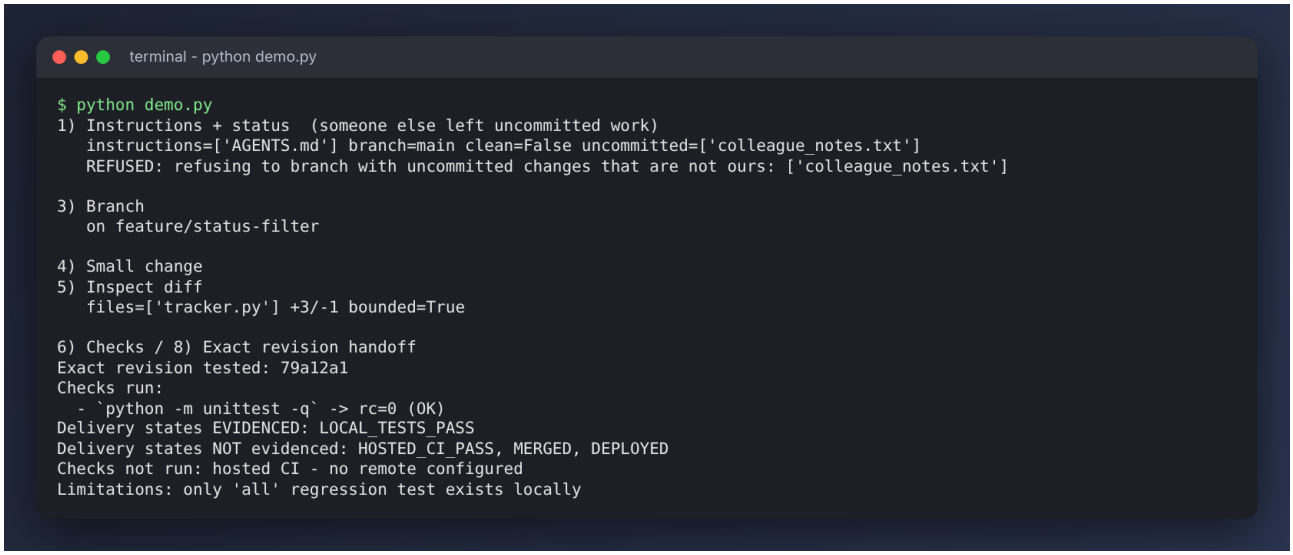
print("\n6) Checks / 8) Exact revision handoff")
checks = run_checks(repo, [[sys.executable, "-m", "unittest", "-q"]])
print(make_handoff(repo, checks, ["hosted CI - no remote configured"], ["only 'all' regression test exists locally"]).r
```

Demo script

## 6. Demo and test screenshots




Test run (real output)




Demo output (python demo.py)

## 7. Evidence-backed report

| | |
|---|---|
| **Change** | Daily Workflow in a Real Repository |
| **Revision tested** | content hash `2e4cd042fb37` of all .py files in this folder (re-run with the git commit SHA after you commit) |
| **Checks run** | `python -m unittest discover` and `python demo.py` |
| **Results** | **PASS** - Ran 6 tests in 0.791s |
| **Checks not run** | Hosted CI, merge and deploy (no remote in the demo) |
| **Known limitations** | Uses a temporary local repo; no push / PR creation is performed. |
| **Next required action** | Human review, then CI on the pull request |

## 8. Practical takeaway

Every handoff states the exact revision and exact checks - evidence belongs to the revision it ran on.